

مشروع مساق بناء المترجمات (CS 351)

مشروع بناء مترجم لغة البرمجة العربية ومحرر الأكواد المستقل

Arabic Programming Language Compiler (Modern C++20) & Standalone IDE

بناء المترجمات (CS 351) — الفصل الدراسي الأول — 1447 هـ (2025/2026م)

المساق الدراسي:

الدكتور القدير / خالد الكحصنة

إشراف أستاذ المقرر:

الأستاذ المهندس / عقيل الشرفي

إشراف مدرس العملي:

1. يعقوب خالد محمد علي المهاجري

إعداد أعضاء المجموعة

2. سليمان صالح صالحي العربي

(المجموعة 20):

3. مالك عادل عبده جبران

الفصل الأول: فكرة المشروع وهندسته المعمارية

1.1 أهداف المشروع ونطاق العمل

يمثل هذا المشروع تطبيقاً هندسياً وميدانياً لمفاهيم بناء المترجمات (Compiler Construction) وتصميم لغات البرمجة. يهدف المشروع إلى تصميم وبناء نظام برمجي متكامل قادر على معالجة نصوص مكتوبة بلغة برمجة عربية فصحي عالية المستوى، وترجمتها بدقة وتنفيذها محلياً.

تم بناء نواة المترجم بالكامل بالاعتماد على معايير لغة (C++20) Modern C++ وبشكل يدوي مستقل دون الاستعانة بأي مولدات خارجية مثل Lex أو Yacc (أو ANTLR)، وذلك لترسيخ التطبيق العملي لمراحل الترجمة الكلاسيكية من البداية وحتى التوليد النهائي.

1.2 الفصل المعماري وآلية الاتصال (IPC Architecture)

تم تطبيق مبدأ الفصل الصارم بين بيئة التحرير التفاعلية وبين نواة المترجم المستقلة عبر أنابيب الاتصال بين العمليات (Inter-Process Communication — IPC Pipes):

- تقوم بيئة المحرر بحفظ الكود العربي المكتوب في ملف مصدري مؤقت بترميز UTF-8.
- يتم تشغيل عملية المترجم التنفيذي المستقل CompilerProject_CPP.exe كعملية فرعية مع تمرير مسار الملف المصدري وراية --json.
- يتم توجيه مدخلات المستخدم لتعليمة اقرار عبر مجرى الدخل القياسي (StandardInput) مباشرة للعملية لتفادي أي حجب أو تعليق غير متزامن.
- ينفذ مترجم C++20 كافة المراحل الست بالتتابع، ثم يصدر وثيقة نواتج الترجمة بصيغة JSON. عبر مجرى الإخراج القياسي (StandardOutput).
- تستلم بيئة المحرر نواتج JSON وتفرزها فوراً لعرض جداول الرموز، شجرة الإعراب، جدول المتغيرات، كود TAC، ولغة التجميع x86، والكونسول.

معمارية النظام المعتمدة — الفصل الثام بين المحرر ومترجم ++C المستقل عبر أنابيب IPC القياسية

CompilerProject_CPP.exe

مترجم C++20 المستقل (Phase CLI Compiler-6)

- ✓ خط أنابيب المراحل الست: Lexer → Parser → Semantic.
- ✓ توليد الشفرة الوسيطة ثلاثية العناوين (TAC Engine).
- ✓ مفسر الذاكرة الافتراضية وتشغيل الكود الفوري.
- ✓ توليد لغة التجميع x86 بتنسيق MASM القياسي.
- ✓ إنتاج الملف التنفيذي المباشر المستقل (output.exe).
- ✓ إدارة جدول الرموز وفحص الأنواع وحماية الثوابت.

file.arb --json

مسار الملف والأوامر →

← حزمة JSON الموحدة

Result DTO

LanguageEditor.exe

محرر الأكواد والواجهة الرسومية (GUI / RTL IDE)

- ✓ واجهة مستخدم رسومية متكاملة داعمة للبرمجة (RTL).
- ✓ تلوين ترميزي للأكواد وتحليل فوري لحظي للأخطاء.
- ✓ شجرة الإعراب المجردة التفاعلية (AST TreeView).
- ✓ 8 السنة تفاعلية مستقلة لعرض مخرجات المترجم.
- ✓ موجه أوامر متكامل بنمط Visual Studio Console.
- ✓ استعراض واختبار أمثلة لغة البرمجة العشرة بنقرة زر.

قناة الاتصال البيني المباشر (IPC: Inter-Process Communication via Anonymous Pipes)

- يقوم المحرر بحفظ الكود في ملف مؤقت ثم استدعاء المترجم كعملية منفصلة عبر Process.Start مع تمرير --json.
- يتم توجيه مدخلات تعليمة اقرار عبر مجرى StandardInput للعملية لمنع أي تجميد أو حجب غير متزامن.
- يستقبل المحرر حزمة DTO JSON كاملة عبر مجرى StandardOutput وتوزيع النتائج على الألسنة فوراً في عزل تام للذاكرة.

شكل 1. معمارية النظام والفصل الثام بين المحرر ومترجم ++C المستقل عبر أنابيب IPC القياسية.

1.3 لغة البرمجة المعتمدة لمترجم المشروع

تم اعتماد لغة (C++20 — ISO/IEC 14882:2020) Modern C++ كلغة حصرية لبناء نواة المترجم المستقل. وتم تجميع المشروع باستخدام مترجم (Visual Studio 2022) MSVC 19.4x مع تطبيق خيارات التحسين القصوى للأداء (/O2 و /std:c++20 و /utf-8).

1.4 جدول الأدوات والمكتبات البرمجية المعتمدة

جدول 1

نوصيف الأدوات والمكتبات البرمجية المستخدمة في بناء المترجم

الأداة / المكتبة	الإصدار	الغرض المنهجي من الاستخدام	الموضع في المترجم
Template Library (STL)	C++ Standard	C++20 (ISO)	إدارة الذاكرة عبر المؤشرات الذكية، معالجة السلاسل بترميز UTF-8، كافة مراحل المترجم الست
Visual C++ (MSVC)	Microsoft	v19.4x (2022)	تجميع وترجمة كود المترجم إلى ملف تنفيذي أصلي 64-بت مباشر وعالي الكفاءة
Win32 API	Windows Console	10/11 SDK	Windows
			ضبط صفحات الترميز (CP_UTF8) وقراءة مجرى الدخل القياسي
			للأنابيب دون إيقاف الخيوط
Inno Setup Compiler	v6.3.3		حزم المترجم ومحرر الأكواد والأمثلة في حزمة تثبيت مستقلة ذاتية التشغيل
			حزمة التسليم النهائي
			(PREXE-G20.exe)

الفصل الثاني: القواعد النحوية للغة البرمجة وهرم الأسبقية

2.1 قواعد لغة البرمجة بصيغة EBNF الموسعة

تم تصميم القواعد النحوية للصورية للغة البرمجة العربية وفق صيغة باكوس نور الموسعة (EBNF) لتكون قواعد لا لبس فيها (Unambiguous) وتتبع صنف اللغات القابلة للتحليل بالنزول العودي (LL(1)):

EBNF Grammar

كود 1: القواعد النحوية للصورية للغة البرمجة العربية بصيغة EBNF

```

<برنامج> ::= <برنامج> <معرف> ";" <كتلة> "."
<كتلة> ::= [ <قسم_التصريحات> ] <قائمة_الجملة>
<قسم_التصريحات> ::= { <تصريح_ثابت> | <تصريح_متغير> | <تصريح_نوع> }
<تصريح_متغير> ::= <متغير> <معرف> { "&";" <معرف> } ":" <نوع_البيانات> ";"
<تصريح_ثابت> ::= <ثابت> <معرف> "=" <قيمة_ثابتة> ";"
<نوع_البيانات> ::= <صحيح> | <حقيقي> | <منطقي> | <حرفي> | <خيطة_رمزي>
<جملة_شرطية> ::= <إذا> <ثم> <فإن> <جملة> [ <والا> <جملة> ]
<حلقة_كرر> ::= <كرر> <معرف> "=" <تعبير> <إلى> <تعبير> [ <اضف> <تعبير> ] "(" <جملة>
<حلقة_طالما> ::= <طالما> "(" <تعبير> ")" <استمر> <جملة>
<حلقة_اعد> ::= <اعد> <جملة> <حتى> "(" <تعبير> ")" ";"
<تعليلة_قراءة> ::= <اقرأ> | <اقرأ> "(" <معرف> { "&";" <معرف> } "(" <معرف> ";"
<تعليلة_طباعة> ::= <اطبع> "(" <تعبير> { "&";" <تعبير> } "(" <معرف> ";"
  
```

2.2 هرم أسبقية المعاملات الحسابية والمنطقية

هرم أسبقية وترتيب المعاملات الحسابية والمنطقية المعتمدة في المترجم

▲ الأعلى أسبقية تنفيذية (Highest Precedence) — تنفيذ أسبق وتجميع تراكم أولوي ▲

المستوى 1 الأقواس وتجميع التعبيرات: (...) أعلى أولوية لتجاوز الترتيب الافتراضي

المستوى 2 المعاملات الأحادية: + ، - ، ! موجب وسالب أحادي ونفي منطقي

المستوى 3 معامل الأس الرياضي: ^ الرفع لقوة (Power)

المستوى 4 الضرب والقسمة وباقي القسمة والعطف: * ، / ، % ، && الضرب والقسمة وباقي القسمة والعطف

المستوى 5 الجمع والطرح والبدل المنطقي: + ، - ، || الجمع والطرح والبدل المنطقي

المستوى 6 معاملات المقارنة والعلاقات: == ، < ، > ، <= ، >= معاملات المقارنة والعلاقات

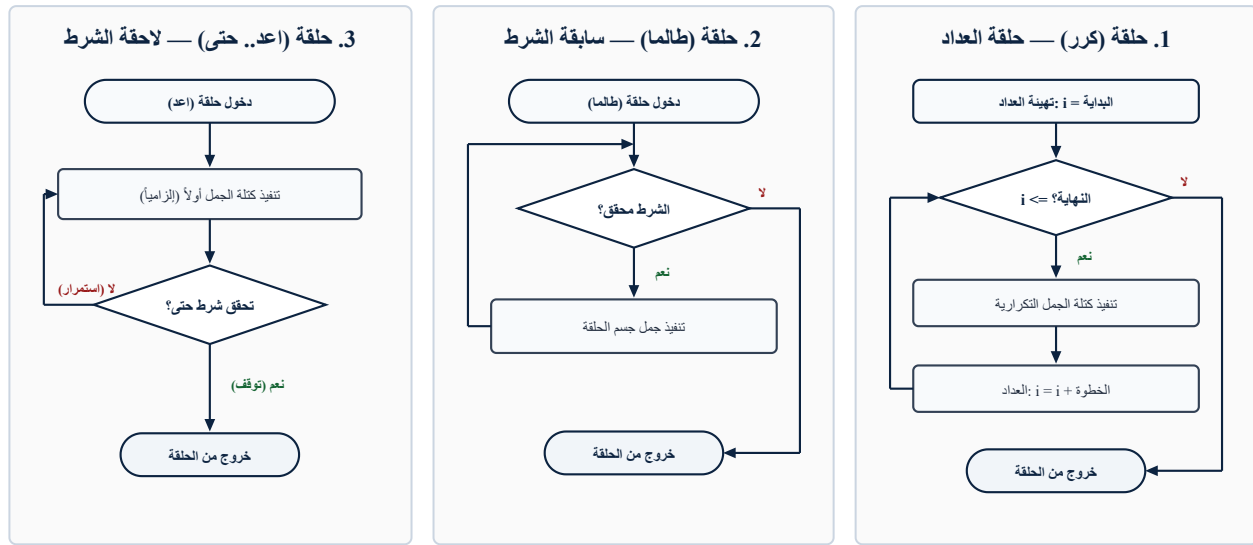
المستوى 7 معامل الإسناد: = تقييم الطرف الأيمن كاملاً

▼ الأدنى أسبقية تنفيذية (Lowest Precedence) — اتجاه تراجع قوة الارتباط التنفيذي ▼

شكل 2. هرم أسبقية وترتيب المعاملات الحسابية والمنطقية المعتمدة في المترجم.

2.3 مخطط تدفق تراكيب حلقات التكرار

مخططات التدفق البرمجي القياسية لتراكيب الحلقات التكرارية الثلاث في لغة البرمجة



شكل 3. مخطط التدفق البرمجي لتراكيب الحلقات التكرارية الثلاث (كرر، طالما، اعد.. حتى).

الفصل الثالث: الشرح التفصيلي لأكواد مترجم ++C وفق العناصر التسعة

يتناول هذا الفصل الشرح التحليلي الصارم لأكواد مترجم Modern C++20 بمراحله الست، حيث يُرفق لكل جزء برمجي مقطع المصدري متبوعاً بالجدول التحليلي للعناصر التسعة المعتمدة: (1) وظيفة الجزء، (2) المدخلات، (3) المخرجات، (4) المتغيرات، (5) هياكل البيانات، (6) الدوال والإجراءات، (7) المكتبات المستخدمة، (8) الملفات المرتبطة، (9) آلية الارتباط بالمشروع.

3.1 نماذج البيانات وعقد شجرة الإعراب (Token.h & Node.h)

Token.h & Node.h

كود 2: نماذج الرموز المعجمية وعقد شجرة الإعراب في Modern C++20

```
struct Token {
    TokenType Type;
    std::string Value;
    int Line = 0;
    Token(TokenType t, std::string v, int l) : Type(t), Value(v), Line(l) {}
};

struct Node {
    std::string Value; // نوع العقدة (IfStatement, ReadStatement, Assign)
    std::string Name; // اسم المعرف
    std::string DataType; // نوع البيانات المصرح به
    std::string Val; // القيمة النصية أو العددية
    int Line = 0; // رقم السطر
    std::vector<std::shared_ptr<Node>> Children; // العقد الفرعية (الأبناء)
    Node(std::string val = "", std::string name = "", int line = 0)
        : Value(val), Name(name), Line(line) {}
};
```

جدول 2

التحليل المنهجي لنماذج البيانات وعقد الإعراب في مترجم ++C

العنصر التحليلي المطلوب	الشرح والتوصيف البرمجي في مترجم ++C
1. وظيفة الجزء	تعريف الكيانات الأساسية للرموز المعجمية (Token) وعقد شجرة الإعراب المجردة (Node) وتوحيد تمثيل البيانات عبر خط الأنابيب.
2. المدخلات (Inputs)	البيانات الأولية المستخلصة من النص المصدري: نوع الرمز، القيمة النصية بترميز UTF-8، ورقم السطر.
3. المخرجات (Outputs)	كائنات مهيكلة بالمؤشرات الذكية تمثل الرموز المعجمية وأشجار الاشتقاق النحوي القابلة للتسلسل بتنسيق JSON.
4. المتغيرات المستخدمة	Type (نوع الرمز)، Value (قيمة الرمز أو تصنيف العقدة)، Name (اسم المتغير)، Line (رقم السطر)، Children (أبناء العقدة).
5. هياكل البيانات	<code>std::vector<std::shared_ptr<Node>></code> للشجرة متفرعة الأبناء، وسلاسل <code>std::string</code> .
6. الدوال والإجراءات	منشآت الكائنات Token () و Node () ، ودوال الطباعة وتوليد JSON.
7. المكتبات المستخدمة	المكتبة القياسية <code>C++20: <string>, <vector>, <memory>, <sstream></code> .
8. الملفات المرتبطة	Token.h, TokenType.h, Node.h, CompilationResult.h وتعتمد عليها كافة المراحل.
9. الارتباط بباقي الأجزاء	تمثل العقد البياني الموحد (Data Contract) الذي تتداوله كافة مكونات المترجم وتستلمه واجهة المحرر عبر JSON.

3.2 المحلل المعجمي واللغوي (Lexer.h & Lexer.cpp)

Lexer.cpp

كود 3: استخراج الرموز المعجمية والكلمات المحجوزة في C++20

```
const std::unordered_set<std::string> Lexer::Keywords = {
    "برنامج", "ثابت", "نوع", "متغير", "صحيح", "حقيقي", "منطقي", "حرفي", "خيطة رمزي",
    "إذا", "فان", "والا", "طالما", "استمر", "اعد", "حتى", "كرر", "الى", "اضف",
    "اقرا", "اقرأ", "اقرء", "اطبع", "صواب", "خطأ", "صح", "خطا"
};

std::vector<Token> Lexer::Tokenize() {
    std::vector<Token> tokens;
    while (_pos < _src.size()) {
        SkipWhitespaceAndComments();
        if (_pos >= _src.size()) break;
        char c = _src[_pos];
        if (IsArabicLetter(c) || isalpha((unsigned char)c) || c == '_') {
            tokens.push_back(ReadIdentifierOrKeyword());
        } else if (isdigit((unsigned char)c) || IsEasternArabicDigit(PeekUtf8(2))) {
            tokens.push_back(ReadNumber());
        } else if (c == '"') {
            tokens.push_back(ReadString());
        } else {
            tokens.push_back(ReadOperatorOrDelimiter());
        }
    }
    tokens.push_back(Token(TokenType::EndOfFile, "نهاية الملف", _line));
    return tokens;
}
```

جدول 3

التحليل المنهجي للمحلل المعجمي في مترجم C++

العنصر التحليلي المطلوب	الشرح والتوصيف البرمجي في مترجم C++
1. وظيفة الجزء	قراءة الكود المصدري العربي بترميز UTF-8، واستخراج الكلمات المفتاحية والمعرفات وتطبيع الأرقام ودعم كافة أشكال الهمزات في تعليمة اقرا.
2. المدخلات (Inputs)	النص المصدري العربي الكامل (const std::string& sourceCode).
3. المخرجات (Outputs)	مصفوفة متجهة مرتبة من الرموز المعجمية (std::vector<Token>).
4. المتغيرات المستخدمة	مؤشر الموقع pos_، عداد الأسطر line_، النص المصدري src_، ومصفوفة الرموز tokens.
5. هياكل البيانات	جدول تجزئة سريع std::unordered_set<std::string> للكلمات ومصفوفة متجهة std::vector<Token>.
6. الدوال والإجراءات	Tokenize(), ReadIdentifierOrKeyword(), ReadNumber(), ReadString(), SkipWhitespaceAndComments().
7. المكتبات المستخدمة	المكتبة القياسية C++20: <unordered_set>, <vector>, <string>, <cctype>, <sstream>.
8. الملفات المرتبطة	Lexer.h, Lexer.cpp, Token.h, TokenType.h.
9. الارتباط بباقي الأجزاء	يمثل المرحلة الأولى لخط الأنابيب، ويسلم تدفق الرموز مباشرة للمحلل النحوي Parser.cpp.

3.3 المحلل النحوي وبناء شجرة الإعراب (Parser.h & Parser.cpp)

Parser.cpp

كود 4: إعراب تعليمة القراءة والنزول العودي في C++20

```

std::shared_ptr<Node> Parser::ParseReadStatement() {
    int line = Current().Line;
    Advance(); // تجاوز اقرا / اقرأ / اقرء
    bool hasParen = Match("(");

    auto readNode = std::make_shared<Node>("ReadStatement", "", line);
    auto firstVar = ExpectType(TokenType::Identifier, "يجب كتابة اسم المتغير المراد القراءة إليه");
    readNode->Name = firstVar.Value;
    readNode->AddChild(std::make_shared<Node>("Identifier", firstVar.Value, line));

    // دعم قراءة عدة متغيرات متتالية مفصولة بفواصل (، أو ،)
    while (MatchComma()) {
        auto nextVar = ExpectType(TokenType::Identifier, "يجب كتابة اسم المتغير بعد الفاصلة في");
        readNode->AddChild(std::make_shared<Node>("Identifier", nextVar.Value, line));
    }

    if (hasParen) Expect(")", "' يجب إغلاق القوس ' لتعليمة 'اقرا'");
    Expect(";", "' يجب إنهاء تعليمة 'اقرا' بفاصلة منقوطة '؛'");
    return readNode;
}

std::shared_ptr<Node> Parser::ParseIf() {
    int line = Current().Line;
    Advance(); // تجاوز 'إذا'
    Expect("(", "' يجب فتح قوس شرط 'إذا'");
    auto cond = ParseExpression();
    Expect(")", "' يجب إغلاق قوس شرط 'إذا'");
    Expect("فان", "' يجب كتابة كلمة 'فان' بعد شرط 'إذا'");
    auto thenStmt = ParseStatement();
    std::shared_ptr<Node> elseStmt = nullptr;
    if (Match("وإلا")) { elseStmt = ParseStatement(); }
    auto node = std::make_shared<Node>("IfStatement", "", line);
    node->Children.push_back(cond);
    node->Children.push_back(thenStmt);
    if (elseStmt) node->Children.push_back(elseStmt);
    return node;
}

```

جدول 4

التحليل المنهجي للمحلل النحوي في مترجم C++

العنصر التحليلي المطلوب	الشرح والتوصيف البرمجي في مترجم C++
1. وظيفة الجزء	التحقق من مطابقة تدفق الرموز لقواعد EBNF للغة العربية وتطبيق خوارزمية النزول العودي LL(1) وبناء شجرة الإعراب والتعافي بنمط Panic Mode.
2. المدخلات (Inputs)	مصفوفة الرموز المعجمية المستخرجة (<std::vector<Token>).
3. المخرجات (Outputs)	العقدة الجذرية لشجرة الإعراب المجردة (<std::shared_ptr<Node>) وقائمة الأخطاء النحوية.
4. المتغيرات المستخدمة	مؤشر موقع الرمز pos_، مصفوفة الرموز tokens_، ومصفوفة الأخطاء errors_.
5. هياكل البيانات	شجرة ديناميكية هرمية متعددة الفروع عبر <std::shared_ptr<Node>، ومصفوفة سلاسل أخطاء <std::vector<std::string>.
6. الدوال والإجراءات	ParseProgram(), ParseDeclarations(), ParseStatements(), ParseReadStatement(), Expect(), Synchronize().
7. المكتبات المستخدمة	المكتبة القياسية <iostream>، <memory>، <vector>، <string>، <C++20>.
8. الملفات المرتبطة	Parser.h, Parser.cpp, Node.h, Token.h.
9. الارتباط بباقي الأجزاء	ينتج الهيكل الشجري الهرمي للبرنامج الذي يمرر للمحلل الدلالي ثم لمولد الكود الوسيط.

3.4 جدول الرموز وإدارة النطاقات والمراجع (SymbolTable.h & SymbolTable.cpp)

SymbolTable.h

كود 5: جدول الرموز وتتبع المراجع في C++20

```

struct SymbolInfo {
    std::string Name;
    std::string DataType;
    std::string Kind; // متغير، ثابت، إجراء، نوع، حقل
    std::string Value;
    int DeclaredLine;
    std::vector<int> ReferencedLines;
    SymbolInfo(std::string n, std::string d, std::string k, int l, std::string v = "-")
        : Name(n), DataType(d), Kind(k), DeclaredLine(l), Value(v) {}
};

bool SymbolTable::Add(const std::string& name, const std::string& dataType,
                     const std::string& kind, int declaredLine, const std::string& value) {
    if (Contains(name)) return false; // منع تكرار التصريح في نفس النطاق
    _symbols.emplace(name, SymbolInfo(name, dataType, kind, declaredLine, value));
    return true;
}

bool SymbolTable::AddReference(const std::string& name, int line) {
    auto sym = Lookup(name);
    if (!sym) return false;
    if (std::find(sym->ReferencedLines.begin(), sym->ReferencedLines.end(), line) == sym-
        >ReferencedLines.end()) {
        sym->ReferencedLines.push_back(line);
    }
    return true;
}

```

جدول 5

التحليل المنهجي لجدول الرموز في مترجم ++C

العنصر التحليلي المطلوب	الشرح والتوصيف البرمجي في مترجم ++C
1. وظيفة الجزء	تسجيل وحفظ خصائص كافة المعرفات في البرنامج (متغيرات، ثوابت) وتتبع أسطر التعريف وأسطر الاستخدام وفحص النطاق ومنع التكرار.
2. المدخلات (Inputs)	بيانات المعرفات من شجرة الإعراب: الاسم، نوع البيانات، التصنيف (متغير/ثابت)، رقم السطر، والقيمة الأولية.
3. المخرجات (Outputs)	سجل الرمز (SymbolInfo)، ونتائج التحقق والاسترجاع بزمناً ثابت $O(1)$.
4. المتغيرات المستخدمة	جدول التجزئة symbols_، ومعلومات الرمز info، ومصفوفة أسطر المراجع ReferencedLines.
5. هياكل البيانات	جدول تجزئة سريع <code><std::unordered_map<std::string, SymbolInfo></code> .
6. الدوال والإجراءات	Add(), Lookup(), Contains(), AddReference(), IsConstant(), GetAllSymbols().
7. المكتبات المستخدمة	المكتبة القياسية <code><C++20: <unordered_map>, <vector>, <string>, <algorithm></code> .
8. الملفات المرتبطة	SymbolTable.h, SymbolTable.cpp, SemanticAnalyzer.h.
9. الارتباط بباقي الأجزاء	المستودع الدلالي المركزي الذي يستند إليه المحلل الدلالي ومولد لغة التجميع لتخصيص مساحات الذاكرة.

3.5 المحلل الدلالي والتحقق من الأنواع وحماية الثوابت (SemanticAnalyzer.cpp)

SemanticAnalyzer.cpp

كود 6: التحقق الدلالي وفحص توافق الأنواع وحماية الثوابت في ++C20

```

bool SemanticAnalyzer::Analyze(const std::shared_ptr<Node>& root) {
    _errors.clear();
    if (!root) return false;
    _symbolTable.Add(root->Name, "برنامج", root->Line, "-");
    PopulateDeclarations(root);           // 1. توثيق التصريحات
    CheckStatementsAndReferences(root);    // 2. فحص الأنواع وحماية الثوابت
    return _errors.empty();
}

void SemanticAnalyzer::CheckStatement(const std::shared_ptr<Node>& node) {
    if (node->Value == "Assign") {
        auto sym = _symbolTable.Lookup(node->Name);
        if (!sym) {
            _errors.push_back("خطأ دلالي سطر " + std::to_string(node->Line) + ": استخدام متغير غير مصرح به " + node->Name + "");
        } else if (sym->Kind == "ثابت") {
            _errors.push_back("خطأ دلالي سطر " + std::to_string(node->Line) + ": محاولة تعديل قيمة الثابت " + node->Name + " المحمي");
        }
        _symbolTable.AddReference(node->Name, node->Line);
    } else if (node->Value == "ReadStatement") {
        auto sym = _symbolTable.Lookup(node->Name);
        if (sym && sym->Kind == "ثابت") {
            _errors.push_back("خطأ دلالي سطر " + std::to_string(node->Line) + ": لا يمكن القراءة إلى الثابت " + node->Name + "");
        }
        _symbolTable.AddReference(node->Name, node->Line);
    }
}

```

جدول 6

التحليل المنهجي للمحلل الدلالي في مترجم ++C

العنصر التحليلي المطلوب	الشرح والتوصيف البرمجي في مترجم ++C
1. وظيفة الجزء	التحقق من سلامة المعنى المنطقي، كشف المتغيرات غير المصرح بها، منع تعديل الثوابت أو القراءة إليها، ومطابقة توافق الأنواع.
2. المدخلات (Inputs)	شجرة الإعراب المجردة (AST Node) وجدول الرموز (SymbolTable).
3. المخرجات (Outputs)	قيمة منطقية bool توضح سلامة البرنامج، وقائمة الأخطاء الدلالية المكتشفة وتحديث أسطر المراجع.
4. المتغيرات المستخدمة	سجل الأخطاء errors_، مرجع جدول الرموز symbolTable_، والمعرفات المسترجعة sym.
5. هياكل البيانات	مصفوفة نصوص للأخطاء std::vector<std::string>.
6. الدوال والإجراءات	Analyze(), ValidateDeclarations(), CheckStatements(), CheckStatement(), AreTypesCompatible().
7. المكتبات المستخدمة	المكتبة القياسية <vector>, <string>, <memory>, <sstream> في C++20.
8. الملفات المرتبطة	SemanticAnalyzer.h, SemanticAnalyzer.cpp, Node.h, SymbolTable.h.
9. الارتباط بباقي الأجزاء	يمثل بوابة الأمان الفاصلة؛ حيث يمنع مرور أي كود معيب منطقياً لمرحلة توليد الكود الوسيط.

3.6 توليد الشفرة الوسيطة ثلاثية العناوين (IntermediateCodeGenerator.cpp)

IntermediateCodeGenerator.cpp

كود 7: توليد تعليمات الكود الوسيط ثلاثي العناوين TAC في C++20

```

std::string IntermediateCodeGenerator::NewTemp() {
    return "t" + std::to_string(++_tempCounter);
}

std::string IntermediateCodeGenerator::NewLabel() {
    return "L" + std::to_string(++_labelCounter);
}

std::string IntermediateCodeGenerator::GenerateExpression(const std::shared_ptr<Node>& node) {
    if (!node) return "";
    if (node->Value == "BinaryOp") {
        std::string left = GenerateExpression(node->Children[0]);
        std::string right = GenerateExpression(node->Children[1]);
        std::string temp = NewTemp();
        _tac.push_back(temp + " = " + left + " " + node->Name + " " + right);
        return temp;
    } else if (node->Value == "Identifier" || node->Value == "Literal") {
        return node->Name.empty() ? node->Val : node->Name;
    }
    return "";
}

```

جدول 7

التحليل المنهجي لمولد الكود الوسيط في مترجم ++C

العنصر التحليلي المطلوب	الشرح والتوصيف البرمجي في مترجم ++C
1. وظيفة الجزء	تحويل شجرة الإعراب الهرمية إلى شفرة وسيطة خطية ثلاثية العناوين (TAC) مستقلة عن المعالج تستخدم متغيرات مؤقتة ووسوم قفز شرطي.
2. المدخلات (Inputs)	شجرة الإعراب المجردة السليمة دلالية (<std::shared_ptr<Node>).
3. المخرجات (Outputs)	مصفوفة تعليمات الكود الوسيط ثلاثي العناوين (TAC <std::vector<std::string>).
4. المتغيرات المستخدمة	عداد المؤقتات tempCounter (\$t_1, t_2\$)، عداد وسوم القفز labelCounter (\$L_1, L_2\$)، ومصفوفة التعليمات tac_.
5. هياكل البيانات	مصفوفة خطية متتابعة <std::vector<std::string>.
6. الدوال والإجراءات	Generate(), GenerateStatement(), GenerateExpression(), NewTemp(), NewLabel().
7. المكتبات المستخدمة	المكتبة القياسية <string>, <vector>, <memory>, <sstream> في C++20.
8. الملفات المرتبطة	IntermediateCodeGenerator.h/cpp, AssemblyCodeGenerator.h, TACInterpreter.h.
9. الارتباط بباقي الأجزاء	يشكل حلقة الوصل المعمارية التي تسلم تعليماتها لمفسر الذاكرة الحية ولمولد لغة التجميع x86.

3.7 توليد لغة التجميع x86 والتنفيذ الحي (AssemblyCodeGenerator.cpp)

AssemblyCodeGenerator.cpp

كود 8: توليد شفرة لغة التجميع x86 بنمط MASM في C++20

```

std::string AssemblyCodeGenerator::GenerateX86Assembly() {
    std::ostringstream ss;
    ss << "; =====\n";
    ss << "(C++) المولدة من المترجم العربي x86 Assembly شفرة لغة التجميع\n";
    ss << "; =====\n";
    ss << ".386\n.model flat, stdcall\noption casemap:none\n\n";
    ss << ".data\n";
    for (const auto& [var, sym] : _varOffsets) {
        ss << "    " << sym << " DD 0\n";
    }
    ss << "\n.code\nmain PROC\n";
    for (const auto& line : _tac) {
        if (line.rfind("//", 0) == 0 || line.empty()) continue;
        if (line.back() == ':') {
            ss << line << "\n";
        } else if (line.rfind("goto ", 0) == 0) {
            ss << "    JMP " << line.substr(5) << "\n";
        }
    }
    ss << "    ret\nmain ENDP\nEND main\n";
    return ss.str();
}

```

جدول 8

التحليل المنهجي لمولد لغة التجميع x86 في مترجم ++C

العنصر التحليلي المطلوب	الشرح والتوصيف البرمجي في مترجم ++C
1. وظيفة الجزء	توليد كود التجميع x86 بنمط MASM، ومحاكاة وتنفيذ تعليمات TAC في الذاكرة الحية وقراءة مدخلات تعليمية اقر ا فوريا.
2. المدخلات (Inputs)	أسطر الكود الوسيط TAC، ومدخلات المستخدم النصية الموجهة عبر مجرى Stdin.
3. المخرجات (Outputs)	كود لغة التجميع (.asm)، ومخرجات الكونسول النصية الناتجة عن التشغيل الفعلي.
4. المتغيرات المستخدمة	عداد البرنامج pc، جدول متغيرات الذاكرة variables_، وقائمة المدخلات inputTokens_.
5. هياكل البيانات	جدول تجزئة قيم المتغيرات <std::unordered_map<std::string, double> و جدول الوسوم.
6. الدوال والإجراءات	GenerateX86Assembly(), TACInterpreter::Execute(), EvaluatePrint(), PrepareInputTokens().
7. المكتبات المستخدمة	المكتبة القياسية <C++20: <iostream>, <sstream>, <unordered_map>, <string>, <cmath>.
8. الملفات المرتبطة	.AssemblyCodeGenerator.h/cpp, TACInterpreter.h/cpp, main.cpp.
9. الارتباط بباقي الأجزاء	المرحلة الختامية للمترجم؛ تولد مخرجات التجميع ونتائج الكونسول وتطبعها بصيغة JSON لواجهة المحرر.

الفصل الرابع: حزمة البرامج الاختبارية العشرة المعتمدة

يتضمن هذا الفصل النص البرمجي الكامل لحزمة الاختبارات العشرة المعتمدة لمترجم ++C، والتي تغطي كافة القواعد والتراكيب اللغوية من العمليات الحسابية، الشروط، الحلقات، الإدخال/الإخراج، والاختبارات السلبية للأخطاء:

01_حساب_المستطيل.arb	المثال 1: تعريف المتغيرات والعمليات الحسابية الأساسية
<pre> برنامج حساب_المستطيل ؛ متغير الطول , العرض , المساحة : صحيح ؛ } الطول = 15 ؛ العرض = 10 ؛ المساحة = الطول * العرض ؛ اطبع ("مساحة المستطيل تساوي:" , المساحة) ؛ . { </pre>	
الهدف: إثبات صحة التصريح عن المتغيرات وإجراء عمليات الضرب والإسناد والطباعة. المخرجات المتوقعة في الكونسول: مساحة المستطيل تساوي: 150	

02_اسبقية_العمليات.arb	المثال 2: العمليات الحسابية المعقدة وأسبقية المعاملات
<pre> برنامج اسبقية_العمليات ؛ متغير س , ص , ع , نتيجة : صحيح ؛ } س = 20 ؛ ص = 4 ؛ ع = 2 ؛ نتيجة = س + ص * ع - 10 / 2 ؛ اطبع ("النتيجة النهائية هي:" , نتيجة) ؛ . { </pre>	
الهدف: إثبات تطبيق أسبقية الضرب والقسمة قبل الجمع والطرح بدقة تامة. المخرجات المتوقعة في الكونسول: النتيجة النهائية هي: 23	

03_فحص_الدرجات.arb

المثال 3: الجمل الشرطية البسيطة والمركبة (إذا / فان / والا)

برنامج فحص_الدرجات ؛

متغير

الدرجة : صحيح ؛

}

الدرجة = 85 ؛

إذا (الدرجة <= 90) فان {

اطبع ("التقدير: ممتاز") ؛

{ والا }

إذا (الدرجة <= 75) فان {

اطبع ("التقدير: جيد جدا") ؛

{ والا }

اطبع ("التقدير: مقبول") ؛

{ ؛

{ ؛

. {

الهدف: إثبات إعراب الشروط المتداخلة وتوليد قفزات TAC وتنفيذ الفرع الصحيح.

المخرجات المتوقعة في الكونسول: التقدير: جيد جدا

04_مجموع_الاعداد.arb

المثال 4: حلقة العداد التراكمية المحددة (كرر)

برنامج مجموع_الاعداد ؛

متغير

العداد , المجموع : صحيح ؛

}

المجموع = 0 ؛

كرر العداد من 1 إلى 10 اضعف 1 {

المجموع = المجموع + العداد ؛

{ ؛

اطبع ("مجموع الأرقام من 1 إلى 10 هو:" , المجموع) ؛

. {

الهدف: إثبات بناء عقد حلقة كسر وتوليد قفزات الزيادة التلقائية للعداد.

المخرجات المتوقعة في الكونسول: مجموع الأرقام من 1 إلى 10 هو: 55

05_حساب_القوى.arb

المثال 5: حلقة الدوران بالشرط المسبق (طالما ... استمر)

برنامج حساب_القوى ؛

متغير

الاساس , العداد , الناتج : صحيح ؛
}

الاساس = 2 ؛ العداد = 1 ؛ الناتج = 1 ؛

طالما (العداد >= 5) استمر {

الناتج = الناتج * الاساس ؛

العداد = العداد + 1 ؛

{ ؛

اطبع ("2 أس 5 يساوي:" , الناتج) ؛

. {

الهدف: إثبات فحص الشرط المسبق وتكرار الكتلة البرمجية وتحديث المتغيرات.

المخرجات المتوقعة في الكونسول: 2 أس 5 يساوي: 32

06_اختبار_اعد_حتى.arb

المثال 6: حلقة الدوران بالتحقق اللاحق (اعد ... حتى)

برنامج اختبار_اعد_حتى ؛

متغير

قيمة_س : صحيح ؛
}

قيمة_س = 1 ؛

اعد {

قيمة_س = قيمة_س * 2 ؛

{ حتى (قيمة_س <= 16) ؛

اطبع ("القيمة بعد انتهاء الحلقة هي:" , قيمة_س) ؛

. {

الهدف: إثبات تنفيذ جسم الحلقة مرة واحدة على الأقل وفحص شرط الخروج اللاحق.

المخرجات المتوقعة في الكونسول: القيمة بعد انتهاء الحلقة هي: 16

07_مضاعفة_المدخلات.arb

المثال 7: تعليمة القراءة من المستخدم (اقرا) والطباعة

برنامج مضاعفة_المدخلات ؛

متغير

العدد_المدخل , الناتج : صحيح ؛
}

اطبع ("ادخل عددا ليتم مضاعفته:") ؛

اقرا (العدد_المدخل) ؛

الناتج = العدد_المدخل * 2 ؛

اطبع ("ضعف العدد المدخل هو:" , الناتج) ؛

. {

الهدف: إثبات معالجة مدخلات المستخدم من حقل الإدخال عبر Stdin بدون تعليق المترجم.

المدخل الاختباري: 25 ← المخرجات المتوقعة: ضعف العدد المدخل هو: 50

08_النظام_العام. arb

المثال 8: البرنامج المدمج لكافة القواعد والتراكيب

برنامج النظام_العام ؛

ثابت

النسبة_المئوية = 100 ؛

متغير

الرقم , المجموع , الضرب : صحيح ؛

}

الرقم = 5 ؛ المجموع = 0 ؛ الضرب = 1 ؛

إذا (الرقم < 0) فان {

طالما (الرقم < 0) استمر {

المجموع = المجموع + الرقم ؛

الضرب = الضرب * الرقم ؛

الرقم = الرقم - 1 ؛

؛ {

؛ {

اطبع ("المجموع التراكمي:" , المجموع) ؛

اطبع ("المضروب الرياضي:" , الضرب) ؛

. {

الهدف: إثبات تكامل الثوابت والمتغيرات والشروط وحلقات التكرار في برنامج واحد.

المخرجات المتوقعة في الكونسول: المجموع التراكمي: 15 ثم المضروب الرياضي: 120

09_خطأ_نحوي_تجريبي. arb

المثال 9: كشف الأخطاء النحوية (Syntax Errors Negative Test)

برنامج خطأ_نحوي_تجريبي ؛

متغير

س , ص : صحيح ؛

}

س = 10

ص = س + 5 * 5 ؛

إذا (س < 5) {

اطبع (س) ؛

؛ {

. {

الهدف: اختبار مائة المحلل النحوي في كشف نسيان الفاصلة المنقوطة وتعاقب المعاملات ومواصلة الإعراب (Panic Mode).

النتيجة الموثقة: كشف الخطأين النحويين في السطرين 5 و 6 وعرضهما في قائمة الأخطاء التفاعلية.

10_خطأ_د لالي_تجريبي. arb

المثال 10: كشف الأخطاء الدلالية وحماية الثوابت (Semantic Errors Negative Test)

برنامج خطأ_د لالي_تجريبي ؛

ثابت

الحد_الثابت = 50 ؛

متغير

س : صحيح ؛

}

س = 20 ؛

الحد_الثابت = 100 ؛

غير_معرف = س + 5 ؛

اقرا (الحد_الثابت) ؛

. {

الهدف: اختبار المحلل الدلالي في منع تعديل الثوابت ومنع القراءة إليها وكشف المتغيرات غير المصرح بها.
النتيجة الموثقة: رصد محاولة تعديل الثابت 'الحد_الثابت' ورصد المتغير 'غير_معرف' في قائمة الأخطاء الدلالية.

الفصل الخامس: التوثيق التطبيقي المصور ومخرجات تنفيذ المشروع

يوثق هذا الفصل لقطات الشاشة الحية لنظام المترجم وبيئة المحرر التفاعلية أثناء اختبار البرامج وتحليل التراكيب:



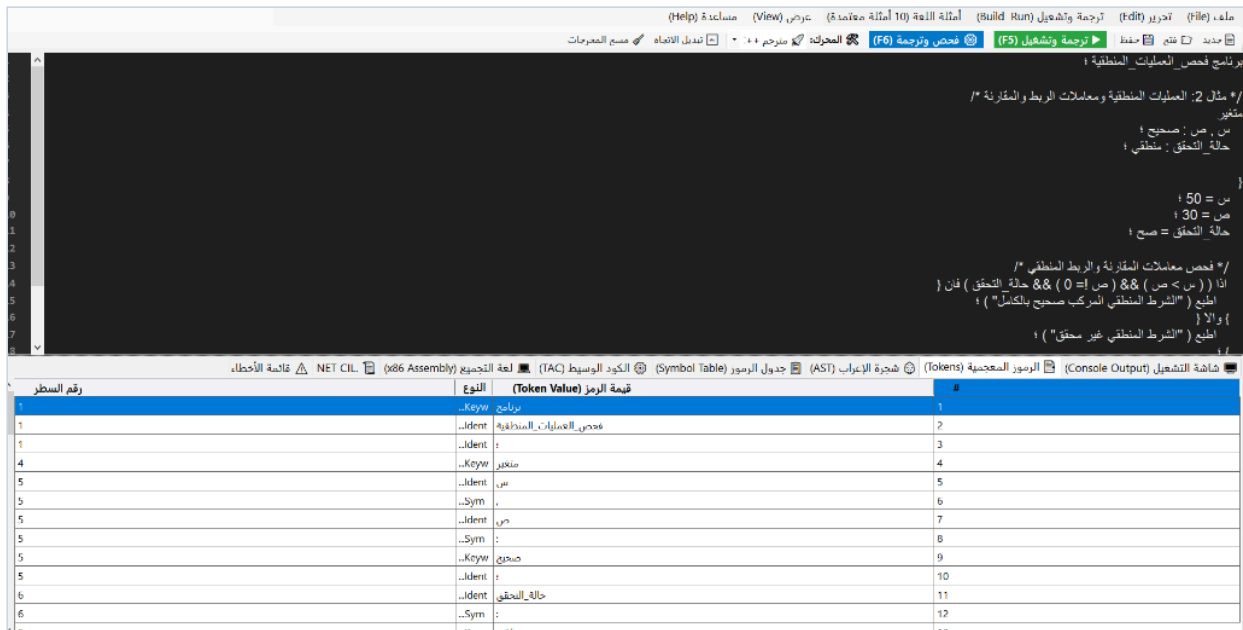
شكل 4. واجهة المحرر التفاعلية مع مخرجات الكونسول وحقل إدخال تعليمة اقرا وزر التحديث الفوري.



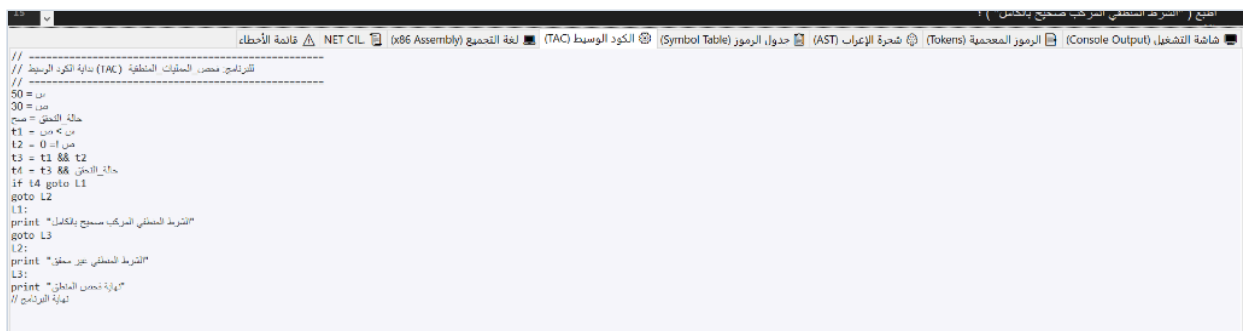
شكل 5. قوائم إدارة الملفات والمشاريع البرمجية في المحرر (جديد، فتح، حفظ، حفظ باسم، خروج).



شكل 6. جدول استخراج الرموز المعجمية (Tokens DataGridView) المصدر من مترجم ++C.



شكل 7. الشجرة المجردة (Abstract Syntax Tree - AST) التفاعلية مع شريط أدوات التحكم البرمجي.



شكل 8. جدول الرموز وإدارة النطاقات والمراجع وتتبع أسطر الاستخدام في مترجم ++C.

شكل 9. الشفرة الوسيطة ثلاثية العناوين (Three-Address Code — TAC) المولدة من مترجم ++C.

#	السطر	نوع الخطأ	وصف تفاصيل الخطأ وموقعه في الكود
1	9	خطأ دلالي (Semantic)	خطأ دلالي في السطر 9: إعادة تعريف المتغير "x".
2	12	خطأ دلالي (Semantic)	خطأ دلالي في السطر 12: لا يمكن تغيير قيمة الثابت "الجواب".
3	13	خطأ دلالي (Semantic)	خطأ دلالي في السطر 13: محاولة الإسناد إلى متغير غير معرف: "متغير مجهول".
4	13	خطأ دلالي (Semantic)	خطأ دلالي في السطر 13: استخدام المتغير غير معرف: "متغير مجهول".

شكل 10. شفرة لغة التجميع x86 بنمط MASM المولدة ذاتيا من مترجم ++C.

```
NET CIL (x86 Assembly) لغة التجميع (Symbol Table) جدول الرموز (AST) شجرة الإعراب (Tokens) الرموز المسجلة (Console Output) شاشة التشغيل
; قائمة الأخطاء
;
; (C++) اللوح من المترجم العربي x86 Assembly كود لغة التجميع
;
.386
.model flat, stdcall
option casemap : none

.data
v_0 DD 0
v_1 DD 0
v_2 DD 0
v_3 DD 0
v_4 DD 0
v_5 DD 0
v_6 DD 0

.code
main PROC
    MOV EAX, 50
    MOV v_0, EAX
    MOV EAX, 30
    MOV v_1, EAX
    MOV EAX, 30
    MOV v_2, EAX
    MOV EAX, 30
    MOV v_3, EAX
    MOV EAX, 0
    MOV v_4, EAX
    MOV EAX, t1 && t2
    MOV v_5, EAX
    MOV EAX, t3 && t4
    MOV v_6, EAX
    ; Cond: t4
    JNE L1
    JMP L2
L1:
    JMP L3
L2:
    JMP L3
```

شكل 11. قائمة تشخيص الأخطاء التفاعلية والتحقق الدلالي والانتقال الفوري لموضع الخطأ.

المراجع والمصادر العلمية المعتمدة (APA 7th Edition)

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). *Compilers: Principles, techniques, and tools* (2nd ed.). Addison-Wesley.
- Al-Kahsah, K. M. (2025). *Compiler construction course project specifications and Arabic grammar rules* (CS 351). Faculty of Computer Science and Applied Sciences, Ibb University.
- Cooper, K. D., & Torczon, L. (2012). *Engineering a compiler* (2nd ed.). Morgan Kaufmann.
- Grune, D., & Jacobs, C. J. H. (2008). *Parsing techniques: A practical guide* (2nd ed.). Springer.
- International Organization for Standardization. (2020). *Programming languages — C++* (ISO/IEC 14882:2020). ISO.
- Irvine, K. R. (2014). *Assembly language for x86 processors* (7th ed.). Pearson.
- Louden, K. C. (1997). *Compiler construction: Principles and practice*. PWS Publishing Company.
- Scott, M. L. (2015). *Programming language pragmatics* (4th ed.). Morgan Kaufmann.
- Unicode Consortium. (2023). *The Unicode standard, version 15.1.0*. <https://www.unicode.org/>